

PushAround: Collaborative Path Clearing via Physics-Informed Hybrid Search

Abstract—The passage of large vehicles in cluttered environments is often blocked by movable obstacles, motivating the use of mobile robot teams to proactively clear traversable corridors. Existing approaches to Navigation Among Movable Obstacles (NAMO) mainly plan abstract sequences of obstacle displacements but often neglect physical feasibility, overlooking robot dimensions, obstacle mass, contact geometry, and required forces, which can lead to strategies that fail in execution. This work introduces *PushAround*, a physics-informed hybrid search framework that incrementally constructs executable clearing plans by coupling geometric connectivity reasoning with short-horizon simulation. Repeated W-clearance connectivity graph (WCCG) queries expose frontier gaps where the vehicle corridor must be widened and rank them by estimated downstream costs, while the search generates direct or recursive pushing candidates – choosing which obstacles to displace, in which directions, and with which contact points and forces – admitting only physics-validated candidates. This interleaved geometry–physics screening avoids exhaustive rollouts over obstacle–action combinations while preserving physically grounded plan expansion. Additional validation characterizes connectivity-query consistency and sensitivity to physics-model mismatch. The framework is validated by extensive simulations and hardware experiments.

I. INTRODUCTION

In many real-world settings, the passage of large vehicles or transport units is obstructed by movable obstacles such as pallets, boxes, or equipment, motivating the deployment of mobile robot teams to actively clear traversable corridors and enable safe passage. This capability is critical in cluttered and unstructured environments such as warehouses, disaster sites, and dense urban spaces. Conventional path planning methods assume static obstacles and thus fail once direct routes are blocked [1]. Navigation Among Movable Obstacles (NAMO) has been studied as a means of creating new routes by pushing, pulling, or rotating obstacles [2], yet most methods abstract away physical feasibility. Factors such as robot dimensions, obstacle size and mass, contact geometry, applied forces, and the coupled dynamics of pushing are typically ignored, producing plans that are *geometrically valid but physically unrealizable*. As shown in Fig. 1, this difficulty is amplified when a large robot cannot reach obstacle boundaries, while a small robot lacks sufficient pushing force. Cooperative pushing by several small robots is a viable alternative, but introduces strong physical coupling since simultaneous pushes can trigger chained motions of multiple obstacles, making prediction and planning substantially more complex.

A. Related Work

NAMO extends motion planning to cluttered environments by planning obstacle displacements through pushing, pulling, or rotating [2], [3]. Later work considers heuristic search, learning-based local planning, physics-based rollouts, and



Fig. 1. Collaborative path clearing in both simulation (**Top**) and hardware (**Bottom**). Two robots actively push movable obstacles to create a W-clear path for a large vehicle to traverse from start to goal, via a hybrid search over the sequence of obstacles to push and the associated pushing modes.

collaborative pushing [3], [4], [5], [6], [7]. However, many NAMO formulations still simplify robot geometry, actuation limits, object mass, and contact dynamics, leading to physically unrealizable plans.

Collaborative pushing has been widely studied in multi-robot manipulation: foundational work on the mechanics of pushing and limit surfaces [8], [9], cooperative transport with robot swarms [10], multi-robot box pushing with reinforcement learning [11], collaborative pushing of polytopic objects in cluttered scenes [7], and learning-based locomanipulation [12]. These studies focus on a prescribed object rather than multi-object corridor clearing.

Physics-informed planning incorporates dynamics models, contact reasoning, or simulation feedback into motion generation: learned dynamics for humanoid contact planning [13], multi-contact force control [14], demonstration-guided optimal control over contact modes [15], physics simulation interleaved with multi-agent pathfinding [16], [17], and learned physical priors [18], [19]. However, they do not directly address collaborative path clearing with many interacting movable obstacles.

B. Our Method

As shown in Fig. 2, this work introduces **PushAround**, a physics-informed framework for collaborative multi-robot pushing that actively constructs traversable corridors for large vehicles in cluttered environments. Its central novelty is a hybrid search that jointly determines which obstacles to displace and how to push them, coupling the high-level sequence of obstacle clearing with the low-level pushing modes that specify contact points, directions, and forces. The framework integrates three components: a W-clearance connectivity graph (WCCG) that certifies vehicle passage under width constraints, a gap-ranking strategy that prioritizes frontier gaps by estimated effort, and pushing-mode generation under geometric

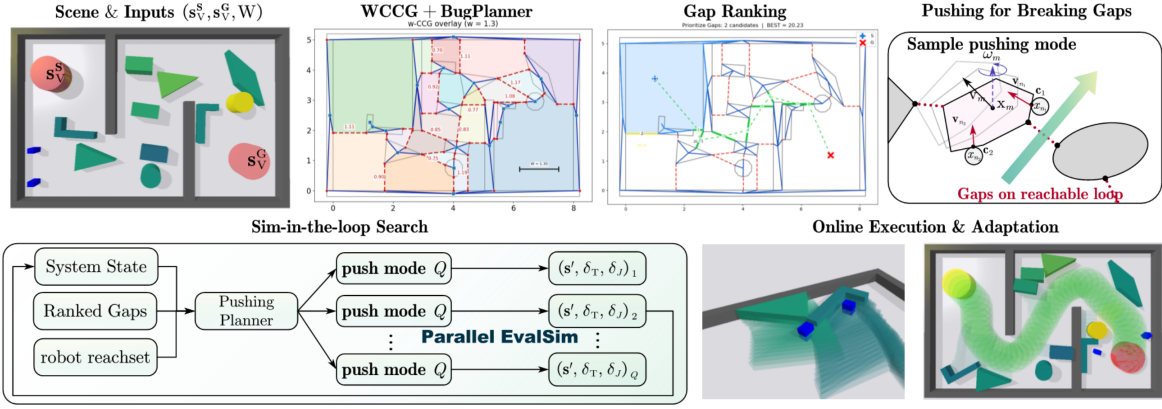


Fig. 2. Overview of the proposed framework. **Top:** Scene inputs, WCCG construction, gap ranking, and sampling of pushing modes for breaking frontier gaps; **Bottom-left:** Sim-in-the-loop hybrid search with parallel validation in simulation; **Bottom-right:** Online execution and adaptation during path clearing.

constraints. These are encapsulated in a search that incrementally expands candidate clearing plans while validating their physical realizability, so the resulting plans are not only geometrically valid but also feasible under robot dimensions, object mass, contact geometry, and coupled pushing dynamics. Since physics validation depends on model fidelity, robustness under execution-model mismatch is also evaluated.

The contributions are twofold: (I) it introduces a unified framework that couples collaborative pushing with physics-informed verification, producing executable plans for path clearing in cluttered environments; and (II) it shows significant improvements in feasibility, efficiency and scalability over existing NAMO and collaborative clearing methods.

II. PROBLEM DESCRIPTION

A. Workspace and Robots

The workspace is a bounded planar region $\mathcal{W} \subset \mathbb{R}^2$ that contains two types of obstacles. A set $\mathcal{O} \subset \mathcal{W}$ represents immovable structures, while a set of $M \geq 0$ movable rigid polygons is defined as $\Omega \triangleq \{\Omega_1, \dots, \Omega_M\} \subset \mathcal{W}$. Each movable obstacle Ω_m is characterized by its mass M_m , inertia I_m , frictional parameters, and state $\mathbf{s}_m(t) \triangleq (\mathbf{x}_m(t), \psi_m(t))$, where $\mathbf{x}_m(t) \in \mathbb{R}^2$ is the planar position of its centroid and $\psi_m(t) \in \mathbb{R}$ its orientation. The region occupied by Ω_m at time t is denoted $\Omega_m(t)$. Moreover, a small team of N robots, indexed as $\mathcal{R} \triangleq \{R_1, \dots, R_N\}$, operates as a cooperative unit. Each robot R_i is modeled as a rigid body with state $\mathbf{s}_{R_i}(t) \triangleq (\mathbf{x}_{R_i}(t), \psi_{R_i}(t))$, where $\mathbf{x}_{R_i}(t) \in \mathbb{R}^2$ is its position and $\psi_{R_i}(t) \in \mathbb{R}$ its orientation. The footprint of R_i is denoted $R_i(t)$. The instantaneous free space is given by:

$$\widehat{\mathcal{W}}(t) \triangleq \mathcal{W} \setminus (\mathcal{O} \cup \{\Omega_m(t)\}_{m=1}^M \cup \{R_i(t)\}_{i=1}^N), \quad (1)$$

where $\widehat{\mathcal{W}}(t)$ excludes regions occupied by immovable obstacles, movable obstacles, and robots.

B. External Vehicle and Clearance Goal

An external vehicle $\mathcal{V}$ whose footprint is enclosed by a disk of radius $W/2 > 0$ must navigate within the workspace from a start position $\mathbf{x}_V^S \in \mathcal{W}$ to a goal position $\mathbf{x}_V^G \in \mathcal{W}$. Since movable obstacles may obstruct the way, the clearance goal is to rearrange them so that the workspace contains a passage

from $\mathbf{x}_V^S$ to $\mathbf{x}_V^G$ that can accommodate this enclosing disk, i.e., a passage of width at least W . At time t , this is expressed by the admissible reference-center space $\mathcal{F}_W(t) \triangleq \{p \in \widehat{\mathcal{W}}(t) \mid \mathbb{B}_{W/2}(p) \subset \widehat{\mathcal{W}}(t)\}$, where $\mathbb{B}_{W/2}(p)$ is the disk of radius $W/2$ centered at p . The W -clearance condition is then given by:

$$\exists \mathcal{P}_V^W \subset \mathcal{F}_W(t) : \mathbf{x}_V^S \rightsquigarrow \mathbf{x}_V^G, \quad (2)$$

which certifies a collision-free centerline path for any vehicle footprint contained in the enclosing disk.

C. Collaborative Pushing Modes

To enlarge blocked gaps, the robots may actively reconfigure Ω by pushing movable obstacles. For obstacle $\Omega_m \in \Omega$, a pushing mode is denoted by $\xi_m \triangleq (\mathbf{c}_m, \mathbf{u}_m)$, where $\mathbf{c}_m = (c_1, \dots, c_N)$ is an ordered tuple of contact points on $\partial\Omega_m$, and the ordering specifies the robot-to-contact assignment. The vector $\mathbf{u}_m \in \mathbb{R}^{2N}$ stacks the corresponding planar contact forces, expressed in the body frame of Ω_m . For a contact tuple $\mathbf{c}_m$, the set of admissible forces $U_m(\mathbf{c}_m) \subset \mathbb{R}^{2N}$ collects contact forces satisfying the robot pushing limits, robot-object friction, and unilateral contact constraints. The body wrench induced by a mode is defined as:

$$\mathbf{q}_m(\xi_m) \triangleq G_m(\mathbf{c}_m)\mathbf{u}_m, \quad (3)$$

where $G_m(\mathbf{c}_m) \in \mathbb{R}^{3 \times 2N}$ is the contact-to-wrench mapping; and $\mathbf{q}_m(\xi_m) \in \mathbb{R}^3$. The admissible mode set is state dependent as denoted by $\Xi_m(\mathbf{s})$. It contains modes whose contacts are reachable from the current robot configurations, pre-contact configurations are feasible in the current clutter, and forces satisfy $\mathbf{u}_m \in U_m(\mathbf{c}_m)$. Hence, applying a feasible mode $\xi_m \in \Xi_m(\mathbf{s}(t))$ yields the physical transition:

$$\mathbf{s}(t^+) \triangleq \Phi(\mathbf{s}(t), m, \xi_m), \quad (4)$$

where $\mathbf{s}(t)$ stacks all robot and obstacle states. The force vector $\mathbf{u}_m$ serves as a feasibility variable to certify wrench generation under the modeled contact and actuation constraints rather than a directly commanded control input.

D. Problem Statement

The goal is to compute a *hybrid schedule* that reconfigures movable obstacles to admit a W -feasible path for the external vehicle. The schedule is defined as:

$$\pi \triangleq \{(\Omega_{m_k}, \xi_k, \Delta t_k)\}_{k=1}^K, \quad (5)$$

where $\Omega_{m_k} \in \Omega$ is the manipulated obstacle, $\xi_k \in \Xi_{m_k}$ is its pushing mode, and $\Delta t_k > 0$ is the execution duration. The search is guided by

$$\mathcal{J}_{\text{clr}}(\pi) \triangleq T + \alpha \sum_{k=1}^K J(m_k, \xi_k, s(t_k)), \quad (6)$$

where $T \triangleq \sum_{k=1}^K \Delta t_k$ denotes the robotic clearance duration excluding subsequent vehicle traversal, and $J(\cdot)$ is the simulation-estimated manipulation effort. Vehicle traversal is considered only through the terminal W -clearance condition (2). The constraints (1)–(4) enforce valid pushing transitions. Equation (6) ranks feasible schedules; PIHS uses finite sampling and does not guarantee global optimality of the continuous hybrid objective.

Remark 1. The objective and constraints above couple obstacle selection, pushing-mode generation, and switching in one hybrid planning problem, yielding a combinatorial-continuous search space far richer than classical NAMO formulations with simple objects or hand-crafted contact modes [8], [10], [11]. ■

III. PROPOSED SOLUTION

As shown in Fig. 2, PushAround first uses the W -Clearance Connectivity Graph in Sec. III-A and gap ranking in Sec. III-B to identify and prioritize blocking gaps. It then generates and evaluates physically feasible collaborative pushing modes in Sec. III-C. These components are integrated into the simulation-in-the-loop hybrid search in Sec. III-D, followed by the online execution procedure summarized in Sec. III-E.

A. W -Clearance Connectivity Graph

The feasibility of routing the vehicle reduces to whether a W -width corridor connects $\mathbf{x}_v^S$ and $\mathbf{x}_v^G$. A W -clearance connectivity graph (WCCG) is introduced here to encode obstacle adjacencies under clearance W and enable fast connectivity queries. Constructed directly from obstacle geometry rather than a workspace grid, the WCCG does not introduce a grid-resolution parameter in the repeated connectivity queries used during planning.

1) *Graph Construction:* The WCCG is built by decomposing all clearance-relevant obstacle and workspace boundaries into convex components or segments. From each component C , a centroid node v_c is created. When two components C_u and C_v have closest points $p_u \in C_u$ and $p_v \in C_v$ with distance less than W , bridge nodes are added at p_u and p_v . These bridge nodes are connected by a bridge-bridge edge, annotated with the gap width $w_{uv} \triangleq \|p_u - p_v\|$, and further linked back to their centroids with centroid-bridge edges. Narrow passages with $w_{uv} < W$ are marked as potential bottlenecks. The resulting graph is given by:

$$\mathcal{G}_W \triangleq (\mathcal{V}, \mathcal{E}_c \cup \mathcal{E}_b), \quad (7)$$

where $\mathcal{V}$ are centroid and bridge nodes; $\mathcal{E}_c$ contains centroid-bridge edges; and $\mathcal{E}_b$ for bridge-bridge edges with widths w_{uv} .

2) *Connectivity Criterion:* Once $\mathcal{G}_W$ has been constructed, connectivity queries can be performed without explicitly computing a geometric path. Let $\text{Face}(\mathcal{G}_W)$ denote the set of connected planar faces induced by the embedded WCCG, i.e., the maximal regions separated by obstacle components and WCCG bottleneck edges. The criterion for (2) is given by:

$$(\mathbf{x}_v^S, \mathbf{x}_v^G \in \mathcal{F}_W(t)) \text{ and } (\exists F \in \text{Face}(\mathcal{G}_W) : \mathbf{x}_v^S, \mathbf{x}_v^G \in F), \quad (8)$$

The criterion follows the standard configuration-space inflation interpretation: expanding obstacles by a disk of radius $W/2$ turns gaps narrower than W into overlapping barriers. Non-convex obstacles are handled through clearance-relevant decomposition, after which WCCG records the separating bottlenecks. The consistency between WCCG connectivity and continuous and rasterized configuration-space references is further evaluated in Sec. IV-A3. A frontier-tracing procedure, similar to the BugPlanner [20], is then used to extract a skeleton Σ as an ordered sequence of centroid and bridge nodes. Starting from $\mathbf{x}_v^S$, it casts a ray toward $\mathbf{x}_v^G$ and follows the encountered frontier loop until either a valid exit is found or a blocking cycle is returned.

B. Ranking of Frontier Gaps

When the condition in (8) fails, the vehicle cannot reach $\mathbf{x}_v^G$ from $\mathbf{x}_v^S$ through a W -clear path $\mathcal{P}_v^W$. In this case, the planner must prioritize *blocking gaps* on the reachable frontier of $\mathcal{G}_W$ according to their predicted downstream cost, and feed the ranked list to the hybrid search as local clearance targets.

1) *Frontier Extraction and Candidate Gaps:* A BugPlanner query on $\mathcal{G}_W$ above either confirms connectivity or returns a counter-clockwise frontier loop $\mathcal{L}$ that separates the two endpoints. The bridge-bridge edges on $\mathcal{L}$ define the first-hop candidate set $\Gamma_{\mathcal{L}} \triangleq \{g_1, \dots, g_K\}$, where each gap g_k is represented by its two closest boundary points, its width $w_{g_k} < W$, and the adjacent obstacles that can potentially be moved. These candidates are then ranked by predicted downstream cost and passed to the hybrid search as local clearance targets.

2) *Evaluation for Immediate Cost:* Each candidate $g \in \Gamma_{\mathcal{L}}$ is evaluated by combining the cost for the robots to reach the gap and the effort required to widen it. Let $\mathbf{s}_{\mathcal{R}}$ be the current robot positions, and $\mathbf{o}_g$ as the outside insertion point of gap g . The resulting one-hop cost is given by:

$$\mathcal{C}(g \mid \mathcal{L}, \mathbf{s}_{\mathcal{R}}) = \lambda_t \mathcal{C}_t(\mathbf{s}_{\mathcal{R}}, \mathbf{o}_g) + \lambda_p (\mathcal{C}_{\text{dp}}(g) + \mathcal{C}_{\text{rp}}(g)) \quad (9)$$

where $\lambda_t, \lambda_p > 0$ weight transition and pushing costs, respectively; $\mathcal{C}_t$, $\mathcal{C}_{\text{dp}}$, and $\mathcal{C}_{\text{rp}}$ denote transition, direct-pushing, and recursive-pushing costs.

3) *Long-term Cost w.r.t. Goal:* Furthermore, to prioritize gaps closer to the goal, a heuristic is added to the evaluation, i.e., $h(g) = \eta \|\mathbf{o}(g) - \mathbf{x}_v^G\|$, with the scaling constant $\eta > 0$. Lastly, a short A*-style presearch is adopted to cross each candidate gap, recompute the next frontier, and continue for a limited width and depth. The *cost-to-connect* is predicted by:

$$\widehat{\text{Cost}}(g) \triangleq \mathcal{C}(g \mid \mathcal{L}, \mathbf{s}_{\mathcal{R}}) + \sum_{g' \in \Pi^*(g)} \{\mathcal{C}(g' \mid \cdot) + h(g')\}, \quad (10)$$

where $\Pi^*(g)$ is the sequence of subsequent gaps discovered after virtually crossing g . Consequently, the final output of this

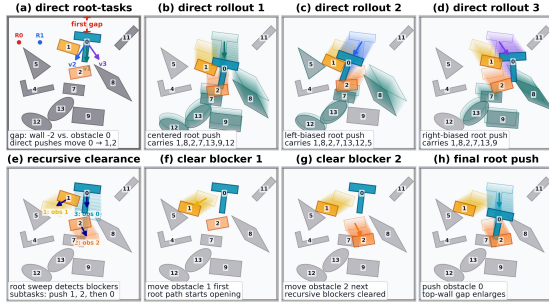


Fig. 3. **Direct** pushing moves a gap-side blocker along a gap-opening direction, whereas **recursive** pushing first clears movable blockers in the swept region and then returns to the root gap-opening task.

module is the ranked list of candidate gaps, i.e., $\text{Rank}(\Gamma_{\mathcal{L}}) \triangleq \text{argsort}_{g \in \Gamma_{\mathcal{L}}} \widehat{\text{Cost}}(g)$, in ascending predicted cost. This ordered set is passed to the hybrid search module, such that only the promising gaps are expanded first.

C. Pushing Mode Generation and Evaluation

For a selected frontier gap g , this subsection constructs a set of candidate pushing strategies $\Xi_g \triangleq \{(\Omega_m, \mathbf{v}, \xi)\}$, where $\mathbf{v} \triangleq (v_x, v_y, \omega) \in \mathbb{R}^3$ is a short-horizon pushing direction for the movable obstacle Ω_m induced by g , and $\xi \triangleq (\mathbf{c}_m, \mathbf{u})$ is the associated pushing mode in (4).

1) *Direct and Recursive Pushing Tasks*: For *direct push*, let Ω_m denote one of the two movable obstacles adjacent to g . For each such obstacle, a small set of candidate body velocities $\{\mathbf{v}_m^k\}_{k=1}^{K_m}$ is sampled with a bias toward enlarging the gap g , while a few angular components are included to account for local geometry. However, if the swept region of a direct push intersects movable blockers, *recursive push* tasks are further required. Namely, for each out-layer blocker Ω_m , the nominal direction moves it away from both the swept region and the reference gap sequence $\Pi^*(g)$, with the candidate velocities $\{\mathbf{v}_m^k\}_{k=1}^{K_m}$. The recursive pushes aim to move the blocker out of the swept region rather than directly open the gap. Recursion therefore creates a bounded dependency chain which prioritizes the final blocker Ω_m , as shown in Fig. 3.

2) *Pushing Mode Generation*: At a search state s , a pushing mode is generated for each candidate direction $\mathbf{v}_m^k$ and obstacle Ω_m . In this subsection, contact tuples, object velocities, forces, and wrenches are expressed in the body frame of Ω_m . For a contact tuple $\mathbf{c}$, its multi-directional force feasibility is:

$$J_{\text{MF}}^m(\mathbf{c}, \mathbf{v}_m^k) \triangleq \sum_{\mathbf{v}_d \in \mathcal{D}_m^k} w_d \min_{\mathbf{u} \in U_m(\mathbf{c})} \|\mathbf{q}_m(\mathbf{c}, \mathbf{u}) + \mathbf{q}_m^\mu(\mathbf{v}_d)\|, \quad (11)$$

where $\|\cdot\|$ is a weighted ℓ_1 wrench norm, the finite set $\mathcal{D}_m^k$ perturbs $\mathbf{v}_m^k$, and $\mathbf{q}_m^\mu(\mathbf{v}_d)$ is the quasi-static friction wrench; the inner minimization is a small LP over the admissible forces $U_m(\mathbf{c})$. A state- and direction-dependent contact set $\mathcal{A}_m^k(s) \subset \partial\Omega_m$ is drawn from the robot-reachable boundary of Ω_m by velocity-force compatibility filtering (Fig. 4), keeping a contact only if its robots reach collision-free pre-contact poses. Starting from cached layouts or random initializations, the tuple is refined by sequential greedy replacement: at the

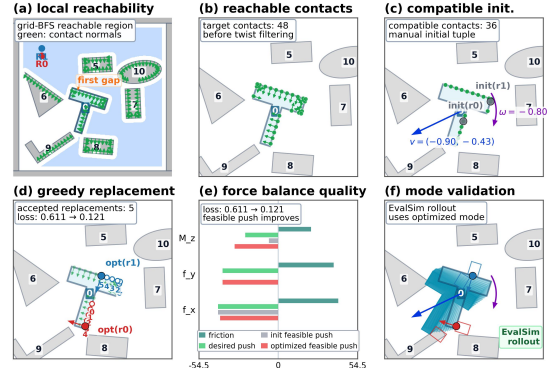


Fig. 4. Reachable contacts are filtered by the desired object twist, assigned to robots, refined by local replacement, and finally validated by simulation.

ℓ -th step, $\mathbf{c}_{r,c}^\ell$ replaces the r -th contact with c , and the best replacement is:

$$c^* = \underset{c \in \mathcal{A}_m^k(s)}{\text{argmin}} J_{\text{MF}}^m(\mathbf{c}_{r,c}^\ell, \mathbf{v}_m^k), \quad (12)$$

swept over contact slots until J_{MF}^m no longer decreases; a bounded backtracking variant removes the robot-order sensitivity of this replacement. If no full-team assignment exists for a task, e.g., on small faces or under the pairwise pusher-clearance constraint, one robot is released for that single push (charged a penalty when ranking modes, so full-team modes are preferred) and the full team is available again afterwards. The best tuple over initializations gives $\mathbf{c}_m^{k,*}$, from which $\mathbf{u}_m^k$ follows from the inner minimization in (11), yielding $\xi_m^k = (\mathbf{c}_m^{k,*}, \mathbf{u}_m^k)$; each mode is cached as a warm start. The best few admissible pairs are kept for gap g and passed to simulation; $\mathbf{u}_m^k$ certifies wrench feasibility while hardware tracks the contact geometry from pose/velocity commands.

3) *Pushing Strategy Evaluation*: For a current state s and a candidate strategy $(\Omega_m, \mathbf{v}, \xi) \in \Xi_g$, a short-horizon rollout evaluates the transition in (4) under the selected mode. As shown in Fig. 4, a strategy is accepted only if the robots reach the pre-contact poses, maintain stable contact, and induce obstacle motion above threshold. The simulation returns:

$$(s', \delta_T, \delta_J) \triangleq \text{EvalSim}(s, (\Omega_m, \mathbf{v}, \xi)), \quad (13)$$

where s' is the successor state capturing the resulting robot and object configurations; $\delta_T > 0$ is the time cost; and δ_J is the control effort. The rollout jointly simulates all robots and movable bodies, thereby capturing contact-induced object interactions. For efficiency, the pushing rollout may be bypassed when the mode has been validated before and the swept region of Ω_m is collision-free.

D. Physics-Informed Hybrid Search

The proposed physics-informed hybrid search (PIHS) selects blocking gaps and validates corresponding multi-robot pushing strategies through short-horizon simulation, as shown in Fig. 5 and summarized in Alg. 1. Unlike purely geometric NAMO planners, PIHS expands only physics-validated strategies through parallel candidate evaluation. It is a feasibility-oriented heuristic search, where Eq. (6) defines the ranking objective for candidate schedules. The search operates on a finite sampled candidate set and therefore does not guarantee

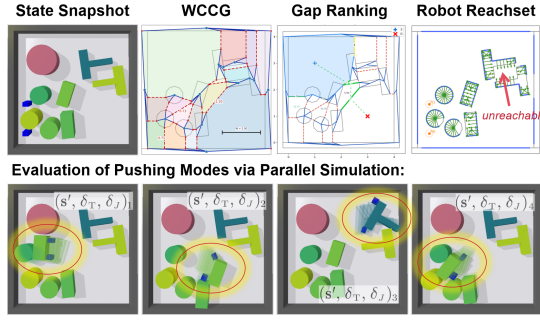


Fig. 5. **Top:** Core components of the proposed physics-informed hybrid search; **Bottom:** Sampled push tasks are evaluated via parallel simulation, producing updated states for search expansion.

global optimality or completeness of the underlying continuous hybrid planning problem.

1) *Tree and Initialization:* The search tree $\mathcal{T}$ is composed of nodes $\nu \triangleq (s, \pi)$, where s is the current system state of all robots and movables in (4), and π the partial pushing strategy in (6). Two global functions are maintained: $\text{Rank}(\nu)$ stores ranked candidate gaps $\Gamma_{\mathcal{L}_\nu}$ given the frontier loop $\mathcal{L}_\nu$ associated with node ν , together with their exploration costs by (10); and $\chi(\nu)$ returns the cumulative execution cost from the root to ν . The root is initialized as $\nu_0 \triangleq (s_0, \emptyset)$, where s_0 is the initial system state.

2) *Node Selection:* At each iteration, the node with minimum best-first priority is selected from the queue. Priority balances realized cost and estimated remaining effort:

$$f(\nu) \triangleq \chi(\nu) + \min_{g \in \text{Rank}(\nu)} \widehat{\text{Cost}}(g), \quad (14)$$

where $\chi(\nu)$ is the cumulative execution cost so far, and the second term is the minimum predicted remaining cost among the unexplored gaps of ν . The remaining-cost term is a heuristic and is not required to be admissible. The score prioritizes nodes by realized cost and promising future actions, but it does not provide an A* optimality guarantee.

3) *Node Expansion with Parallel Simulation:* Let the selected node be $\nu \triangleq (s, \pi)$. For each ranked gap $g \in \text{Rank}(\nu)$, its predicted gap sequence $\Pi^*(g)$ in (10) is first derived. Then, the mode optimization by (11) provides a candidate strategy set $\Xi_g \triangleq \{(\Omega_m, \mathbf{v}, \xi)\}$, including both the direct push of the adjacent movable obstacles, and recursive push of out-layer obstacles. Thus, expansion is prioritized first by the gap ordering in $\text{Rank}(\nu)$, and then by the candidate ordering within Ξ_g . Each candidate strategy $(\Omega_m, \mathbf{v}, \xi) \in \Xi_g$ is first checked by a geometric quick-pass and, if it passes, evaluated by the short-horizon rollout in (13). A successful evaluation yields (s', δ_T, δ_J) , from which a child node $\nu' \triangleq (s', \pi')$ is created by appending $\tau \triangleq (\Omega_m, \mathbf{v}, \xi)$ to the current partial schedule π . The cumulative cost is updated by: $\chi(\nu') \triangleq \chi(\nu) + \widehat{C}(\nu, \tau)$, where $\widehat{C}(\nu, \tau) \triangleq \delta_T + \alpha \delta_J$ is the realized incremental cost. Note that all candidate strategies in the current batch are evaluated *concurrently in parallel*, and a single expansion can produce multiple validated successors.

4) *Termination:* Termination occurs when a node ν reaches a state s for which a W -clear path $\mathcal{P}_V^W$ exists from $\mathbf{x}_V^S$ to $\mathbf{x}_V^G$ by (8). Thus, the schedule π stored in ν is a feasible solution satisfying the terminal clearance condition and the validated

Algorithm 1: Physics-Informed Hybrid Search

Input: $s_0, \alpha, \text{EvalSim}(\cdot), W$ -criterion by (8)
Output: π^*

```

1  $\nu_0 \leftarrow (s_0, \emptyset), \chi(\nu_0) = 0;$ 
2 Init  $\mathcal{Q}$  by (14);
3 while not terminated do
4    $\nu \leftarrow \mathcal{Q}.\text{pop\_min}()$  by (14);
5   foreach  $g \in \text{Rank}(\nu)$  do
6     Compute  $\Pi^*(g)$ , generate  $\Xi_g$ ;
7     foreach  $(\Omega_m, \mathbf{v}, \xi) \in \Xi_g$  do
8        $\tau = (\Omega_m, \mathbf{v}, \xi)$  // Direct or recur.
9        $(s', \delta_T, \delta_J) \leftarrow \text{EvalSim}(\nu, \tau)$  by (13);
10      if success then
11         $\nu' \leftarrow (s', \pi \cup \{\tau\});$ 
12         $\chi(\nu') \leftarrow \chi(\nu) + \delta_T + \alpha \delta_J;$ 
13        Add  $\nu'$  to  $\mathcal{Q}$ ;
14      if  $s'$  admits  $\mathcal{P}_V^W$  by (8) then
15         $\pi^* \leftarrow \pi',$  break;
16 return  $\pi^*;$ 

```

pushing transitions. It encodes both the sequence of gaps to clear and the pushing strategies that realize them, without implying that $\mathcal{J}_{\text{clr}}(\pi)$ is globally minimal.

Remark 2. Efficiency comes from three lightweight heuristics: gap-lookahead reduces unproductive branches, MODE-CACHE warm-starts mode optimization, and the quick-pass rollout bypass skips simple motions whose mode was previously validated, avoiding redundant simulations. ■

E. Overall Analyses

1) *Execution and Online Adaptation:* The hybrid search returns a pushing schedule $\pi = \{\tau_1, \dots, \tau_K\}$, with $\tau_k \triangleq (\Omega_k, \mathbf{v}_k, \xi_k)$. The schedule is executed in a receding-horizon manner: for each τ_k the robots reach the mode ξ_k and follow the pushing direction $\mathbf{v}_k$ from the current state, as in (13). On the hardware the optimized contact forces certify feasibility rather than being commanded; the controller converts the planned contact geometry and object motion into velocity commands with feedback tracking. During transitions, robots are reassigned by preserving the circular order of contact points along the obstacle boundary [7], reducing path crossings under sufficient clearance. After each push the state is updated from observation, the WCCG is rebuilt from the current configuration, and execution continues until a W -clear path $\mathcal{P}_V^W$ connects $\mathbf{x}_V^S$ and $\mathbf{x}_V^G$. Replanning is triggered by observed execution discrepancies – failed contact establishment, stalled pushing, or invalidated WCCG connectivity – in which case Alg. 1 is called from the current state to derive the updated schedule.

2) *Computational Complexity:* Given M movable obstacles, the computational cost of one node expansion in Alg. 1 is dominated by candidate generation and physics validation. The WCCG update and gap ranking incur $\mathcal{O}(M \log M + |\mathcal{V}| + |\mathcal{E}_c| + |\mathcal{E}_b|)$ and $\mathcal{O}(|\Gamma_{\mathcal{L}_\nu}| \log |\Gamma_{\mathcal{L}_\nu}|)$ costs, respectively, while the dominant simulation cost is

$$\mathcal{O} \left(\sum_{g \in \text{Rank}(\nu)} |\Xi_g| \left(C_{\text{LP}} + \frac{\rho_\nu T_{\text{sim}}}{n_w} \right) \right),$$

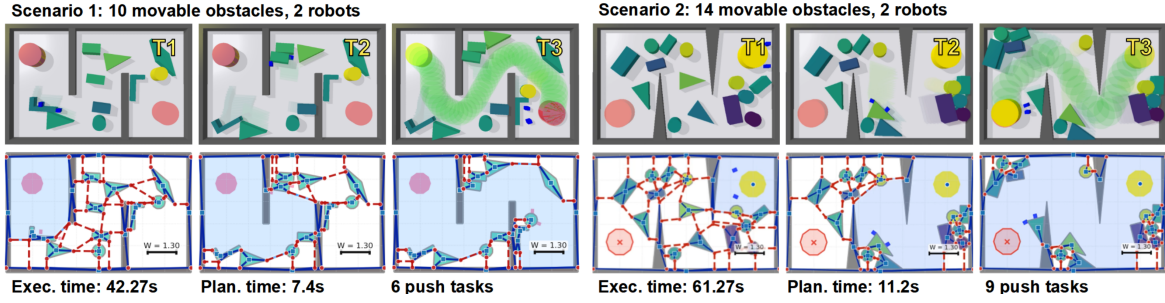


Fig. 6. Execution and planning alignment in Scenario 1 (Left) and Scenario 2 (Right). **Top:** simulated snapshots during path clearing. **Bottom:** corresponding WCCG overlays with the start face in blue. The executed pushes progressively expand the reachable W -clearance region until the goal is connected.

TABLE I
WCCG CONNECTIVITY VALIDATION AND MEAN RUNTIME.

Type	CC=FR	FP/FN [†]	Time (ms)		
			CC	FR	WCCG
Convex	100/100	0/0	1.23	282.9	0.86
Non-convex	98/100	0/0	1.51	204.1	1.30
Mixed	98/100	0/0	1.33	242.4	1.05
Total	296/300	0/0	1.36	243.1	1.07

[†] WCCG errors on CC-FR-consistent cases. All runtimes are measured in one harness over the same 300 scenes, with WCCG served by its compiled backend;

Therefore, scalability is mainly determined by validated pushing candidates, while quick-pass filtering and parallel simulation reduce the practical expansion cost. Robot-team size mainly affects mode generation, where greedy contact refinement avoids exhaustive $|A|^N$ contact assignment and evaluates bounded single-contact replacements.

IV. EXPERIMENTS

Numerical experiments are conducted to validate the proposed PIHS algorithm. The algorithms are implemented in Python3 and PyBullet [21] on a computer with an Intel Core i9-14900KF and an NVIDIA RTX 4080. Videos are provided as multimedia attachments.

A. Numerical Simulations

1) *Setup and Configuration:* The simulation uses a step size of $\Delta t = 1/240$ s and a control period of $1/40$ s. Robots are modeled as disk or box pushers consistent with the W -clearance definition, while movable obstacles have masses uniformly sampled from $[5, 15]$ kg. A trial succeeds when the external vehicle reaches its goal through a W -clear path. The nominal scenario uses an 8×5 m workspace with bottleneck-forming bar obstacles and randomly distributed movable objects. Two robots clear 10–15 movable obstacles under minimum-separation constraints. Unless otherwise stated, comparisons and ablations are conducted on Scenario 1 with 10 movable obstacles over 40 trials, while Scenario 2 with 14 obstacles provides an additional cluttered case. All methods use the same robot footprints, obstacle configurations, collision/reachability checks, and execution simulator, and are given the same per-trial timeout of 800 s and planning budget of 20,000 control steps; unsuccessful trials are assigned the 800 s cap in the planning-time statistics.

2) *Results:* Two representative simulation runs are shown in Fig. 6. In Scenario 1 (10 movable obstacles, two robots) the start and goal belong to different W -clearance faces, so the planner clears the blocked passage with 6 pushes in 7.4 s and

TABLE II
PERFORMANCE COMPARISON AND ABLATIONS ON SCENARIO 1 OVER 40 RANDOMIZED TRIALS.

Method / Variant	Succ.(%)	PT (s)	ET (s)	#Sims	#Pushes
DFS-WCCG	25.0	>100.0	51.2	>500	8.0
SL-Push (offline)	62.5	0.06	44.1	0.0	7.3
SL-Push (sim)	75.0	18.2	64.6	20.0	10.2
Rec-NAMO	37.5	13.3	42.4	0.0	7.8
PushAround (ours)	97.5	10.3	28.6	121.5	6.0
w/o gap-lookahead	82.5	16.9	35.2	178.3	7.2
w/o mode-cache	92.5	17.1	33.4	164.6	6.5
w/o quick-pass	90.0	23.6	41.9	208.2	6.1
w/o reinsertion	85.0	12.8	31.1	151.7	7.8



Fig. 7. Failure cases by baselines Rec-NAMO, SL-Push and DFS-WCCG.

the execution finishes in 42.27 s with the start face containing the goal. Scenario 2 (14 obstacles) requires a 9-push plan in 11.2 s and 61.27 s of execution. In both cases the executed pushes progressively enlarge the reachable W -clearance region until a complete W -clear path is found, as shown by the expanding blue start face in the WCCG overlays.

3) *WCCG Connectivity Validation:* WCCG is compared against continuous C-space (CC) and a rasterized reference at 0.02 m (FR) on 300 random scenarios (Table I): CC and FR agree in 296 cases, WCCG agrees with CC in all 300 with zero FP/FN, and a query costs 1.07 ms against 1.36 ms (CC) and 243 ms (FR), while additionally supplying the gap graph needed by PIHS.

4) *Comparisons:* Four baselines are evaluated on Scenario 1 over 40 randomized trials under the same W -clearance criterion, object distribution, and simulator: **DFS-WCCG** (WCCG-guided search with fixed pushes), **SL-Push** (offline and simulation-validated variants), and **Rec-NAMO** (recursive NAMO) [3], [4], [16], [5]. Table II shows that PushAround reaches the highest success rate (97.5%) and the lowest combined planning-and-execution time (38.9 s). DFS-WCCG needs more than 100 s of planning (exponential branching); SL-Push (offline) plans in 0.06 s but many displacements fail in execution (62.5%); SL-Push (sim) reaches 75.0% at 18.2 s

TABLE III
SCALABILITY AND ROBUSTNESS UNDER OBSTACLE DENSITY,
ROBOT-TEAM SIZE, AND EXECUTION-SIDE PHYSICS MISMATCH.

Case	Trials	Succ.(%)	PT (s)	ET (s)
<i>Obstacle density</i> (PushAround, $N=2$, nominal)				
$M = 15$	10	100	2.5 ± 0.9	37.1 ± 21.0
$M = 30$	10	100	11.7 ± 3.7	100.9 ± 34.7
$M = 45$	10	100	53.4 ± 24.5	272.7 ± 97.4
<i>Robot-team size</i> ($M=30$, nominal; 10 trials per cell)				
PushAround, $N=3$	10	100	30.0 ± 12.7	97.3 ± 42.1
PushAround, $N=4$	10	100	29.2 ± 8.9	110.1 ± 33.1
SL-Push, $N=3$	10	80	30.0 ± 46.3	193.3 ± 111.3
SL-Push, $N=4$	10	80	23.4 ± 18.2	171.7 ± 61.2
DFS-WCCG, $N=3$	10	100	142.6 ± 183.1	178.1 ± 72.1
DFS-WCCG, $N=4$	10	90	101.9 ± 103.9	175.9 ± 113.6
<i>Physics mismatch</i> ($M=30$, $N=2$)				
Nominal	10	100	16.8 ± 3.4	104.7 ± 37.4
Mass $\times 0.6-1.4$	40	100	16.8 ± 5.4	105.9 ± 44.6
Lateral fric. $\times 0.6-1.4$	40	97.5	16.5 ± 5.7	114.0 ± 48.5
Spin fric. $\times 0.6-1.4$	40	97.5	15.2 ± 3.6	100.1 ± 35.7
Force limit $\times 0.6-1.4$	40	95.0	18.9 ± 10.9	115.9 ± 58.5
All perturbed	160	97.5	16.9 ± 7.0	108.9 ± 47.4

Each perturbed physics row scales one execution-side parameter over the factor range shown while the others remain nominal.

planning and 10.2 pushes; Rec-NAMO often fails to build a connected W -clear corridor in dense clutter. Figure 7 shows representative baseline failures.

5) *Ablation Studies*: Four internal ablations are conducted: **w/o gap-lookahead** (frontier gaps ranked without downstream costs), **w/o mode-cache** (no warm start from previously validated modes), **w/o quick-pass** (all candidates evaluated by rollout), and **w/o reinsertion** (temporarily failed nodes not reconsidered). Table II shows that gap-lookahead and reinsertion mainly improve robustness (success drops by more than 10% when either is removed), whereas mode-cache is a warm start and only lowers success to 92.5%. Quick-pass affects efficiency: without it simulations rise from 121.5 to 208.2 and planning from 10.3 to 23.6 s.

6) *Scalability and Robustness Analysis*: Generalization is further evaluated in the same 8×8 m workspace by scaling the obstacle density M , the robot-team size N , and the execution-side physics parameters; Table III summarizes representative settings and Fig. 9 plots the complete density sweep. (1) **Obstacle density**: with two robots and 10 trials per density, PushAround keeps a 100% success rate for all $M \in [15, 45]$ while its mean planning time grows from 2.5 to 53.4 s, whereas all baselines solve none of the $M=45$ trials and removing recursive pushes lowers success to 70% (Fig. 8 shows representative runs). (2) **Robot-team size**: at $M=30$, PushAround solves all ten trials at $N=2, 3$, and 4, after two mode-generation refinements: bounded backtracking contact assignment removes the robot-order sensitivity of greedy replacement, and when no full-team assignment exists for a task, one robot is released for that single push and re-activated afterwards, with a penalty favoring full-team modes. Since the fallback belongs to the shared mode generator, the baselines benefit as well (Table III); PushAround is nevertheless the only method that solves all ten trials at every team size and the most economical, e.g., at $N=4$ with 29.2 s of planning, 110.1 s of execution and 10 pushes versus 101.9 s, 175.9 s and 14 for DFS-WCCG, and 171.7 s of execution and 20 pushes

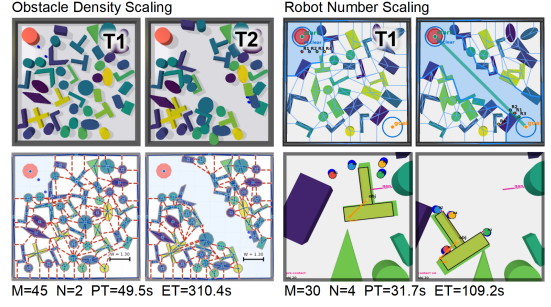


Fig. 8. Scalability of the clearing process. **Left**: obstacle-density scaling ($M=45$ objects, two robots): snapshots during the clearing (**top**) and WCCG overlays with the start face in blue (**bottom**). **Right**: robot-team scaling ($M=30$ objects, four robots): initial and final states (**top**) and zoomed views of one push before contact and during execution (**bottom**).

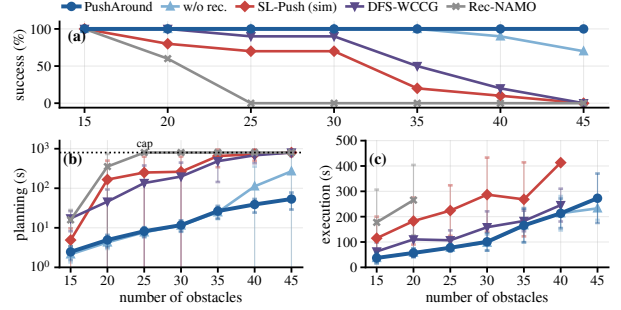


Fig. 9. Obstacle-density scaling (8×8 m, 10 independent seeds per density). (a) Success rate; (b) mean planning time, with unsuccessful trials capped at the 800 s budget (dotted line); (c) mean execution time of the successful trials (Rec-NAMO has none beyond $M=20$). Error bars denote one standard deviation.

for SL-Push (sim). (3) **Physics mismatch**: with planning on the nominal model and only the execution simulator perturbed (one parameter at a time), PushAround succeeds in 156 of 160 perturbed trials (97.5%; nominal 10/10), the four failures being one low-friction planning failure and three budget time-outs. Under the same $\pm 40\%$ perturbations the baselines are far less robust: SL-Push (sim) averages 32.5% success (0% under the strongest friction and force reductions) and DFS-WCCG 57.5% at an order of magnitude higher planning time.

B. Hardware Experiments

1) *System Description*: Experiments use a 5×6 m workspace of interlocking 0.6×0.6 m foam mats (Fig. 10), 2 UGVs and 6 cardboard-box movable obstacles. Robots and movables each carry three to four motion-capture markers whose global poses are streamed to PyBullet in real time. The two Mecanum UGVs are controlled through planar velocity commands, while the optimized contact forces are used only for feasibility evaluation rather than direct motor control. Motion capture provides the current robot and object poses, which are compared with the successor state predicted by EvalSim: a push is deemed to deviate and triggers replanning only when the measured object position error exceeds 0.3 m or its orientation error exceeds $\pi/4$. The threshold is deliberately loose, tolerating a slight over-push or off-axis push. Movables are randomly placed for each trial.

2) *Results*: Two representative real-world runs are shown in Fig. 10. In Scenario 1, PIHS breaks five candidate gaps within 6.5 s using 12 node expansions, 49 visited snapshots

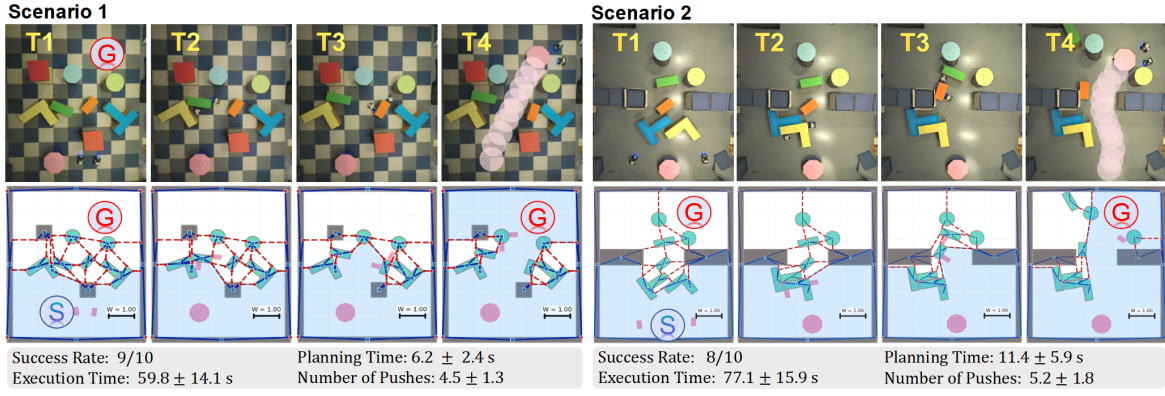


Fig. 10. Real-world pushing experiments with the execution–planning alignment (over 20 runs and 2 scenarios). **Top:** snapshots of two robots pushing obstacles in a 5×6 m workspace with movable objects and fixed boundaries. **Bottom:** WCCG overlays with the start face in blue.

and search depth 6; the four-task schedule first pushes and rotates the yellow L-shaped object to open a gap near an immovable obstacle, then pushes the green and orange objects apart until the green cylinder is pushed to the right and a W -clear path is established at $t = 52$ s. In Scenario 2, PushAround succeeds in 8/10 trials with an average of 5.2 pushes and 11.4 s of planning; it first selects a direct push on the blue L-shaped object, which also moves the adjacent T-shaped block and opens the first gap into the narrow passage. This neighboring-object motion is not a separately encoded action but an emergent outcome of the coupled rigid-body rollout in EvalSim, whose snapshot is used to update the WCCG and continue replanning. The complete execution takes 77.1 s on average with 15.9 s deviations, mainly from direction-dependent force limits, drifting, and Mecanum-wheel slippage near 45° motion directions, which are compensated by online replanning.

V. CONCLUSION

This work presents a multi-robot framework PushAround for path clearing in unstructured environments using a hybrid search algorithm that jointly plans obstacles selection, contact points, and pushing forces efficiently. The method demonstrates hardware feasibility and uses state-feedback replanning to recover from observed execution deviations. Future directions include obstacle states estimation without external sensing, uncertain masses, and limited robot visibility for real-world applications.

REFERENCES

- [1] L. Liu, X. Wang, X. Yang, H. Liu, J. Li, and P. Wang, "Path planning techniques for mobile robots: Review and prospect," *Expert Systems with Applications*, vol. 227, p. 120254, 2023.
- [2] M. Stilman and J. J. Kuffner, "Navigation among movable obstacles: Real-time reasoning in complex environments," *International Journal of Humanoid Robotics*, vol. 2, no. 04, pp. 479–503, 2005.
- [3] M. Stilman, J.-U. Schamburek, J. Kuffner, and T. Asfour, "Manipulation planning among movable obstacles," in *Proceedings 2007 IEEE international conference on robotics and automation*, 2007, pp. 3327–3332.
- [4] L. Yao, V. Modugno, A. M. Delfaki, Y. Liu, D. Stoyanov, and D. Kanoulas, "Local path planning among pushable objects based on reinforcement learning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 3062–3068.
- [5] Z. Ren, B. Suvonov, G. Chen, B. He, Y. Liao, C. Fermuller, and J. Zhang, "Search-based path planning in interactive environments among movable obstacles," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 533–539.
- [6] J. J. Weeda, S. Bakker, G. Chen, and J. Alonso-Mora, "Pushing through clutter with movability awareness of blocking obstacles," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 512–518.
- [7] Z. Tang, Y. Zhang, and M. Guo, "Pushingbots: Collaborative pushing via neural accelerated combinatorial hybrid optimization," *IEEE Transactions on Robotics*, vol. 42, pp. 579–599, 2026.
- [8] S. Goyal, A. Ruina, and J. Papadopoulos, "Limit surface and moment function descriptions of planar sliding," in *IEEE International Conference on Robotics and Automation (ICRA)*, 1989, pp. 794–795.
- [9] K. M. Lynch, H. Maekawa, and K. Tanie, "Manipulation and active sensing by pushing using tactile feedback," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, vol. 1, 1992, pp. 416–421.
- [10] J. Chen, M. Gauci, W. Li, A. Kolling, and R. Groß, "Occlusion-based cooperative transport with a swarm of miniature mobile robots," *IEEE Transactions on Robotics*, vol. 31, no. 2, pp. 307–321, 2015.
- [11] Y. Wang and C. W. De Silva, "Multi-robot box-pushing: Single-agent q-learning vs. team q-learning," in *IEEE/RSJ international conference on intelligent robots and systems*, 2006, pp. 3694–3699.
- [12] Y. Feng, C. Hong, Y. Niu, S. Liu, Y. Yang, and D. Zhao, "Learning multi-agent loco-manipulation for long-horizon quadrupedal pushing," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 14 441–14 448.
- [13] Y.-C. Lin, B. Ponton, L. Righetti, and D. Berenson, "Efficient humanoid contact planning using learned centroidal dynamics prediction," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 5280–5286.
- [14] Q. Rouxel, S. Ivaldi, and J.-B. Mouret, "Multi-contact whole-body force control for position-controlled robots," *IEEE Robotics and Automation Letters*, vol. 9, no. 6, pp. 5639–5646, 2024.
- [15] T. Xue, H. Girgin, T. S. Lembono, and S. Calinon, "Demonstration-guided optimal control for long-term non-prehensile planar manipulation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 4999–5005.
- [16] D. M. Saxena and M. Likhachev, "Planning for complex non-prehensile manipulation among movable objects by interleaving multi-agent pathfinding and physics-based simulation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 8141–8147.
- [17] K. Ren, G. Wang, A. S. Morgan, L. E. Kavraki, and K. Hang, "Object-centric kinodynamic planning for nonprehensile robot rearrangement manipulation," *IEEE Transactions on Robotics*, vol. 41, pp. 5761–5780, 2025.
- [18] R. Ni and A. H. Qureshi, "Progressive learning for physics-informed neural motion planning," in *Robotics: Science and Systems*, 2023.
- [19] Y. Liu, R. Ni, and A. H. Qureshi, "Physics-informed neural mapping and motion planning in unknown environments," *IEEE Transactions on Robotics*, vol. 41, pp. 2200–2212, 2025.
- [20] K. N. McGuire, G. C. H. E. de Croon, and K. Tuyls, "A comparative study of bug algorithms for robot navigation," *Robotics and Autonomous Systems*, vol. 121, p. 103261, 2019.
- [21] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning," <http://pybullet.org>, 2016–2019.